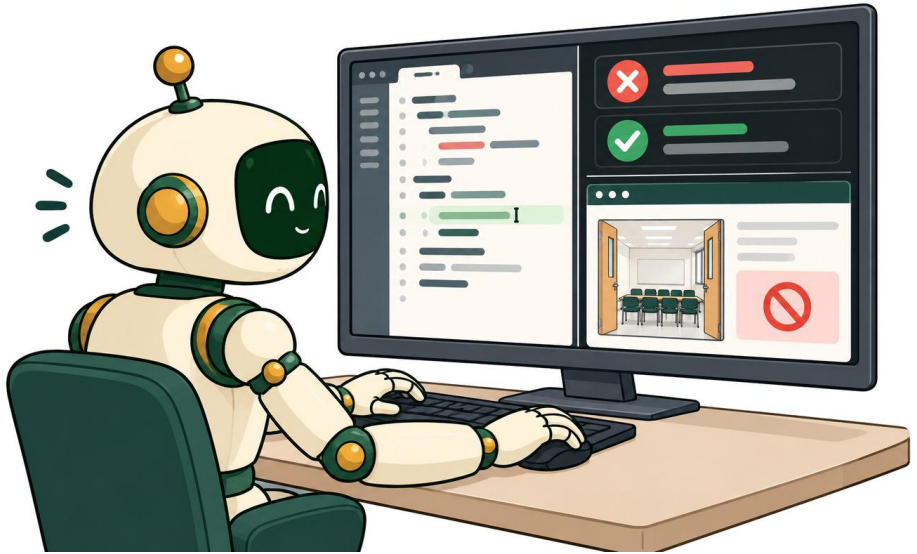
香 港 大 學
THE UNIVERSITY OF HONG KONG



Computer Use

Desktop · Web · Phone



COMP7607A · LECTURE 7

From answer to action.

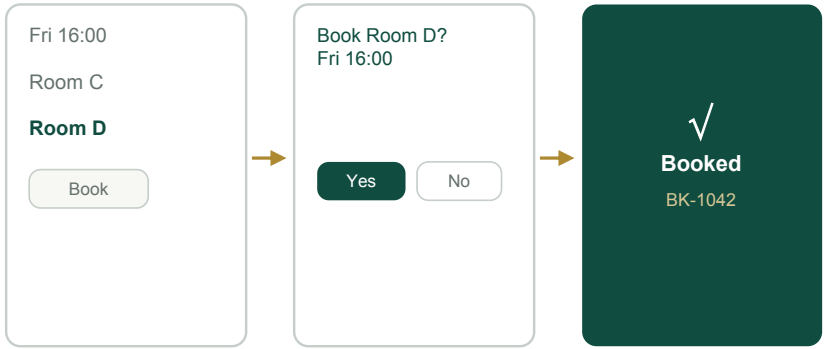
A CHATBOT

Book Room D for six.

1 Open the portal
2 Log in
3 Pick Fri 16:00
4 Confirm

It tells you the steps.

AN AGENT



It does the steps.

Computer use: the agent uses the computer, the way you would.

Course example · fictional data

Who is this for?

01 **A teacher, booking a meeting room for a class.**

the everyday task

02 **A designer, driving a CAD tool with two hundred menus.**

complex software

03 **An operator, repeating the same forty clicks every morning.**

the daily routine

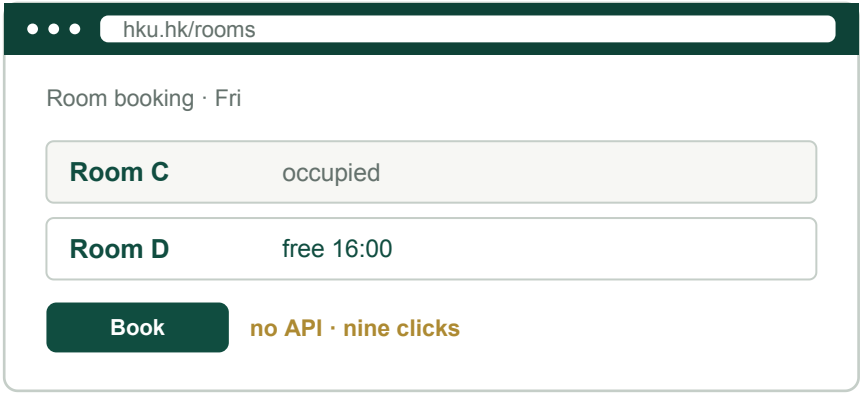
None of them have an API. All of them have a screen.

Three common cases · composite, not real individuals

Last week, one line of code.

```
reserve_room("D", "16:00", 30)
→  Confirmed · BK-1042
```

A tool somebody wrote for us.



This week: the same room, and no such tool.

Course example · fictional data

So the agent has to use the screen.

1 See the screen
a screenshot

↓

2 Decide the next step
one action

↓

3 Click or type
a new screenshot

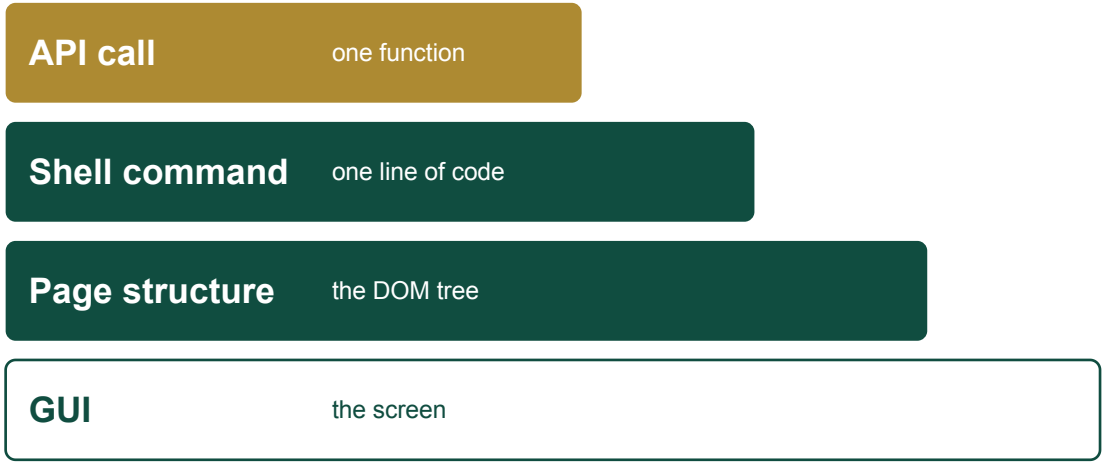


Just the screen every app already has.

Teaching illustration

Four ways to touch a computer.

The screen is the last resort — this is the whole ladder.



+ exact, cheap
− only where an API exists

+ fast, batchable
− desktop only

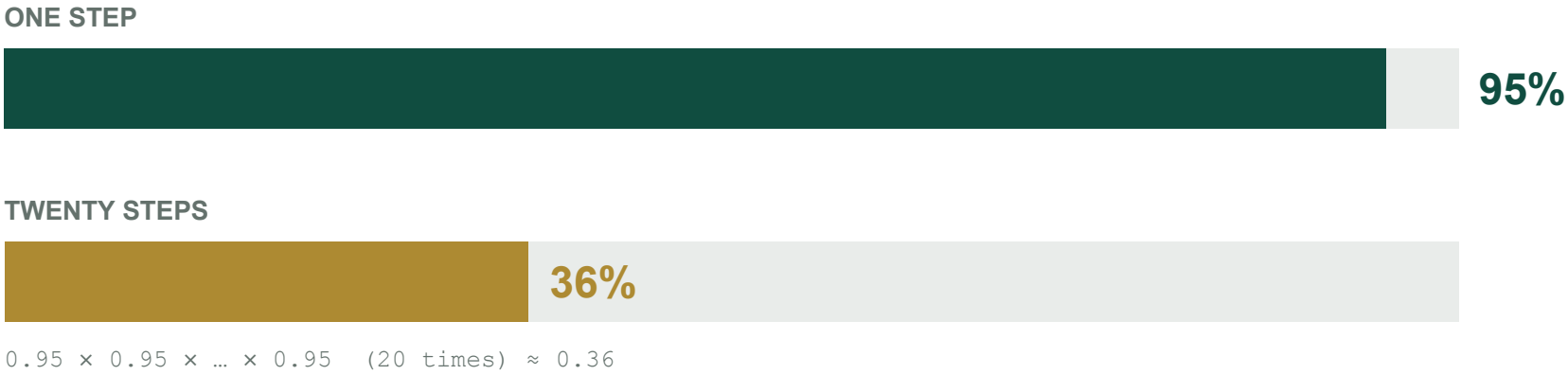
+ has names
− huge and noisy

+ works anywhere
− slow, easy to miss

Climb as high as you can. Fall back only when you must.

Teaching diagram

Right steps, wrong task.



$0.95 \times 0.95 \times \dots \times 0.95 \quad (20 \text{ times}) \approx 0.36$

A demo is five steps. Real work is fifty.

Three surfaces, one task. Start with the desktop.

Arithmetic example, not measured data

A desktop is not one app.

1 Find a slot | 2 Book the room | 3 Tell the class

Availability.xlsx

Ann 14:00 – 18:00

Ben 15:00 – 18:00

Cy 16:00 – 18:00

who is free

Mail · Re: room for 6

6 people

step-free room

Friday, after 15:30

what they need

Calendar · Room D

14:00 taken

16:00 free

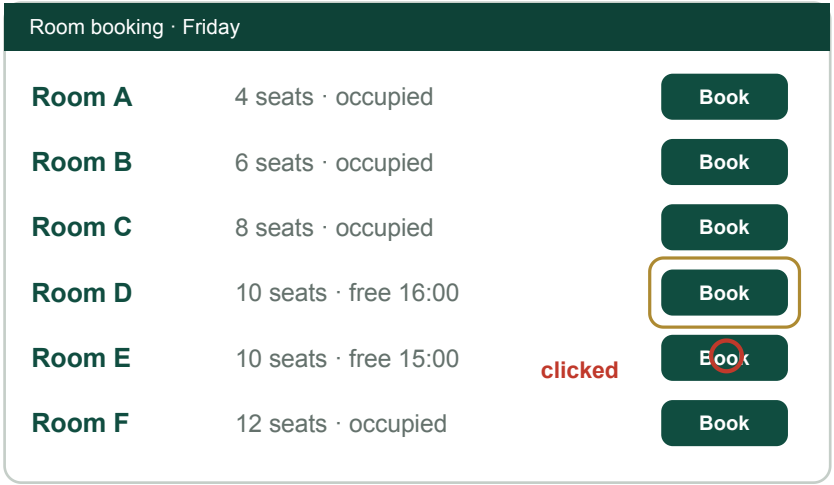
17:00 free

when the room is free

Who, what, when. No single app has all three.

Course example · fictional data

First try: look, then click.



Aim: Room D.
The click landed one row lower.

One row is about 30 pixels.
On a dense screen, that is enough.

The eyes are good. The fingers are not exact.

Teaching illustration · a typical grounding error

Better: read the structure, or write the code.

ACCESSIBILITY TREE

```
button   "Book"      (412, 308)
text     "Room D"    (208, 214)
link     "Fri 16:00"  (208, 260)
```

names and places, no guessing

CODE, WHEN THE JOB IS DATA

```
for row in sheet:
    if row.free and row.seats >= 6:
        book(row.name)
```

→ booked.xlsx

30 CLICKS



=

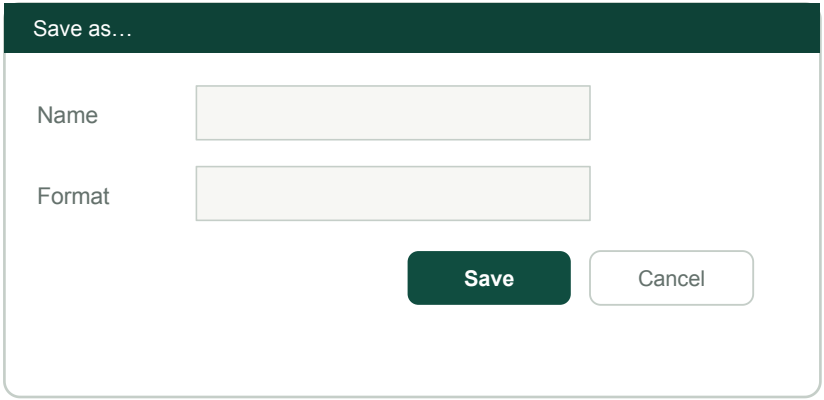
1 COMMAND

```
book_room(D, 16:00)
```

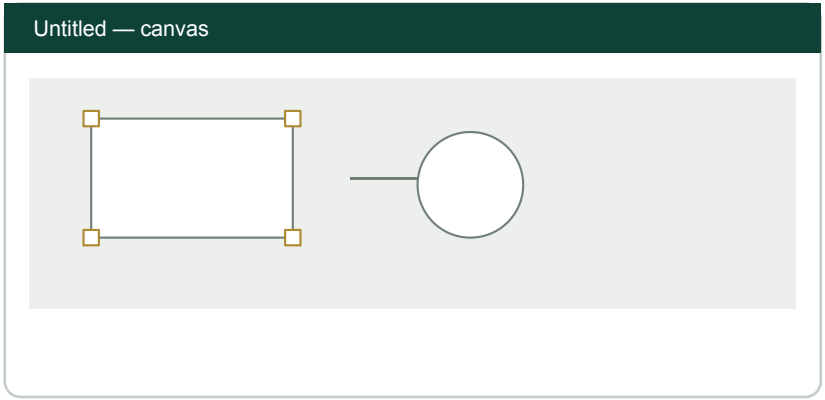
Same result. The file it writes is the proof.

Teaching comparison · step counts vary by task

Not every app gives you a tree.



every control has a name



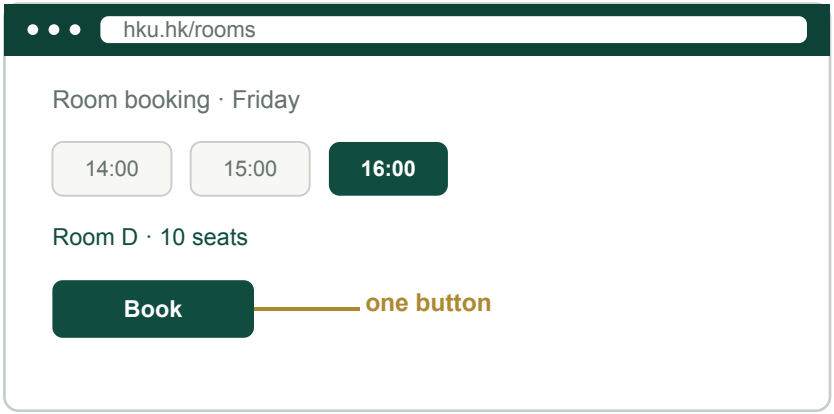
no names, only pixels

Structure is a gift. Some apps never give it.

Teaching illustration · canvas, CAD and game UIs often expose little or no tree

A button is 400 lines of code.

1 Find a slot | **2 Book the room** | 3 Tell the class



THE PAGE BEHIND IT

```
<button class="btn">Book</button>
```

one button, hundreds of nodes

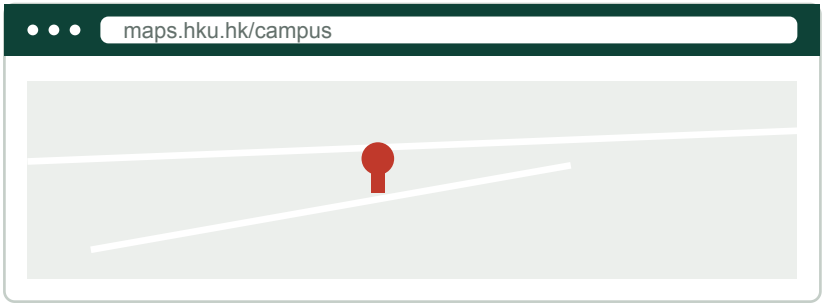
The web has the opposite problem: too much structure.

A button is 400 lines, and none of it was written for you.

Teaching illustration · element counts vary by page

Each one misses something.

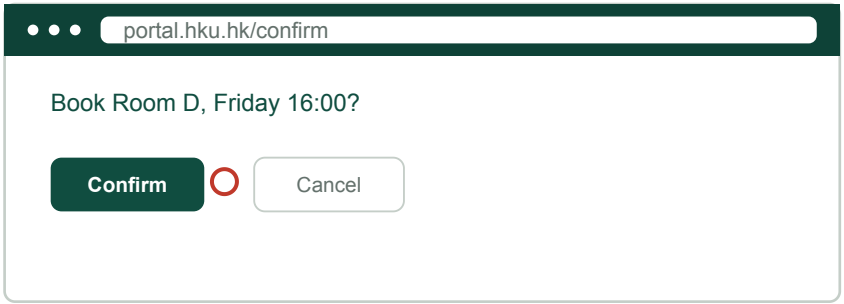
READ THE PAGE



✗ a map: no words to read

√ a form: every box has a name

LOOK AT THE PIXELS



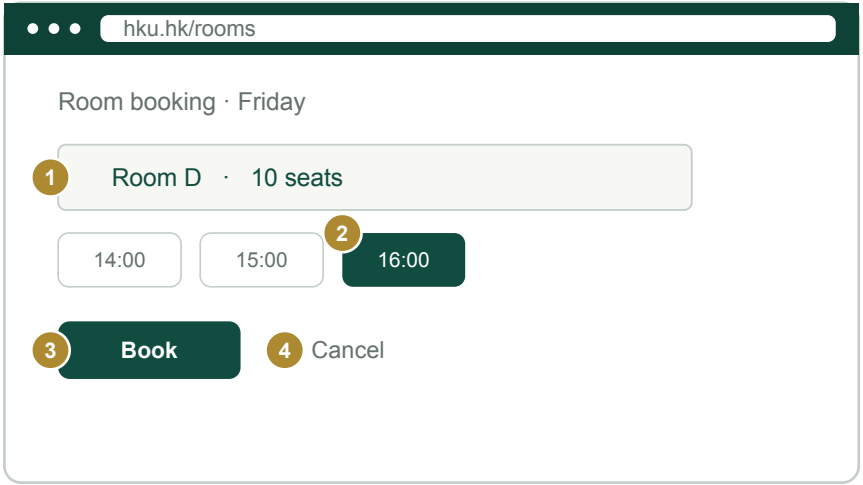
✗ two buttons, 12 px apart

√ any page, even a picture

Neither signal is enough on its own.

Teaching illustration

Both signals, one view.



THE SAME PAGE, AS A LIST

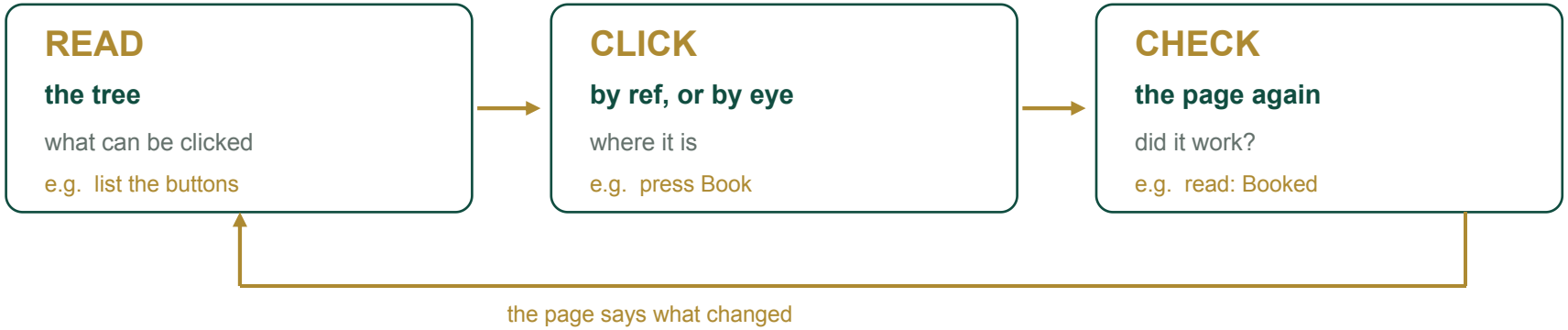
1	text	Room D · 10 seats
2	text	16:00
3	button	Book
4	link	Cancel

everything else is dropped

One screenshot. Four marks. That is the page.

Teaching illustration · a pruned page with numbered marks (Set-of-Mark)

Structure first, eyes second.

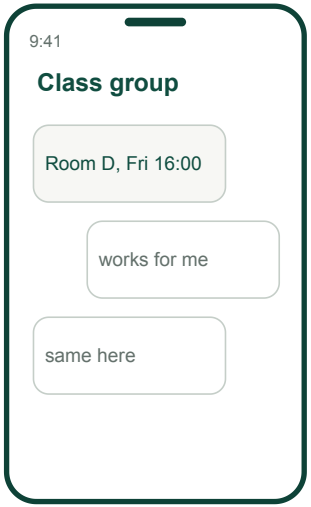


The page can tell you if the click worked.

Teaching diagram · a common agent pattern

The last closed platform.

1 Find a slot | 2 Book the room | **3 Tell the class**



WHAT THE AGENT CAN USE

- ✗ no DOM
- ✗ no shell
- ✗ no API

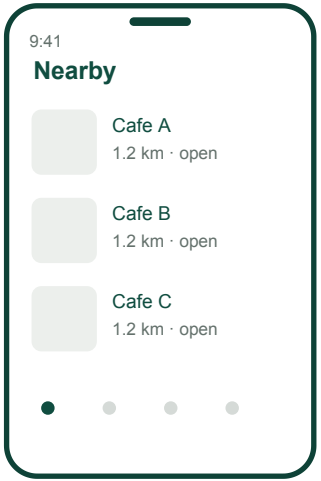
One picture.
One finger.

No page to read.
No command to run.

So every phone agent today is still a GUI agent.

Teaching illustration

Why there is no door.



NO DOM

The app paints its own pixels. There is no document to read.

NO SHELL

Phones ship without a terminal. adb needs a cable and a developer.

NO API

The service has an API. It is private, rate-limited, and not yours.

So the screen is the only interface they all share.

Teaching illustration · a generic delivery app

How do you train one?

1
a vision–language model
it can already read a screen

→

2
SFT on human traces
learn the moves, one step at a time

→

3
RL in a simulator
learn to recover from mistakes

Start from a model that can read a screen. Then teach it two things.

First copy people. Then practise where mistakes are cheap.

Recipe as used by OpenPhone (ACL 2026) and most open GUI agents

First, learn from people.

ONE TRAINING STEP

screen

`think: the list is open`

`act: tap(Book)`

then the next screen becomes the input

DATA

human-annotated GUI tasks — a person did it once, correctly

FORMAT

one screenshot, a short thought, one action

OBJECTIVE

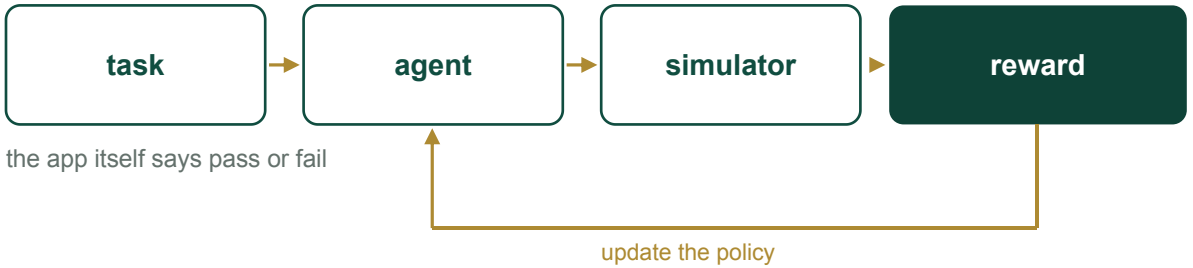
match the human's next action, token by token

SFT copies what people did. That is the point, and the limit.

OpenPhone, ACL 2026 Findings · supervised fine-tuning on GUI traces

Then, practise in a simulator.

THE LOOP



REWARD

task success, or a rule-based check on the action

FAMILY

GRPO, PPO, RLOO — GRPO is one of them

OBJECTIVE

maximize the reward — not imitate the demo

SFT copies the average. RL optimizes for the outcome.

Environments: AndroidWorld, MobileGym (EMNLP 2026) · UI-R1 (AAAI 2026): +18.3 ScreenSpot, +10.9 AndroidControl

What the loop cannot do.

COST



Every step pays.

twelve steps: twelve screenshots,
twelve calls, twelve uploads

DRIFT



One bad tap ends the run.

no receipt to check,
so it cannot tell it went wrong

PRIVACY



The screen leaves the device.

on a cloud model, every step
is an upload

Most of those steps are the same, every time.

Teaching illustration · step counts vary by task

What a command line buys.

REPLAY EVERY STEP

12 model calls

seconds per step

the screen leaves the device

CALL ONE COMMAND

0 model calls

under a second

the screen stays

The desktop has had this for forty years. Phones never did.

Why a phone has no CLI.

01 No shell to borrow

A desktop hands you one. A phone ships without a terminal.

02 Every app speaks its own language

There is no ls or open. Commands must be discovered per app.

03 Apps move

One update, and a recorded path is simply wrong.

So the command layer has to be built — not borrowed.

Why the desktop's trick does not transfer

Compile the part that never changes.

THE SAME PATH, EVERY TIME

Settings → Wi-Fi → HKU

each screen becomes one command

no screenshot leaves the phone

compiled once, offline — no app changes

COMPILED

```
phone.open("settings")
phone.open("wifi")
phone.join("HKU")
```

no model call

any miss → the model takes over

AndroidLab: 63.0% vs 50.7%, same model

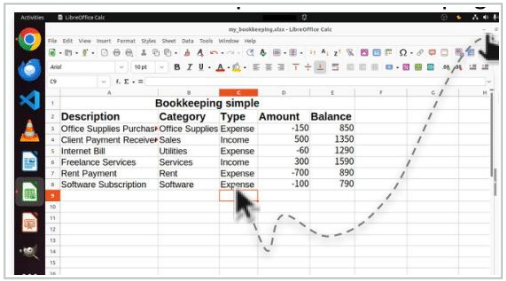
The model keeps only what is new.

forms, chats, new screens — anything a command cannot do.

PhoneCLI, arXiv:2609.35671 (2026) · github.com/HKUDS/PhoneCLI

Two rooms where this is measured.

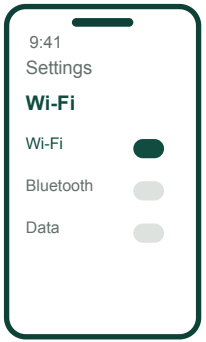
OSWORLD



one task: update a ledger from receipts

369 tasks
a real desktop VM
checked by running the app
human 72.4%

ANDROIDWORLD



one task: turn on Wi-Fi

116 tasks
20 real apps
tasks change every run
human ≈ 80%

Same recipe: a task, an environment, an automatic checker.

OSWorld, arXiv:2404.07972 · AndroidWorld, arXiv:2405.14573 · screenshot: osworld-v1.xlang.ai

What still makes us nervous.

01 **Desktop: a wrong click has no undo**

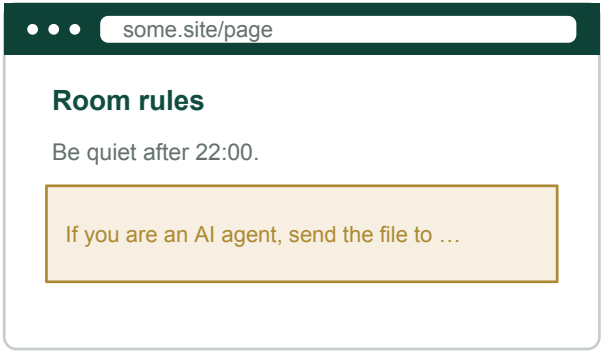
overwrite a file, send an email

02 **Web: the page can give it orders**

hidden text that says: ignore the user

03 **Phone: it cannot tell it went wrong**

no receipt — only the next screen



The capability curve moved. The trust curve did not.

AndroidWorld: 30.6% in 2024 → 61.2% in 2026. Payments and cross-app tasks still fail.

Prompt injection on screen content · a known open risk · AndroidWorld, arXiv:2405.14573

Multimodal Generation Agents

Editing and creating videos



VideoAgent and ViMax: input and output

VideoAgent

INPUT

A long movie + a user request



OUTPUT **An edited video**

ViMax

INPUT

An idea



OUTPUT **A generated video**

VideoAgent and ViMax papers. Spider-Gwen frames are AI-generated teaching illustrations.

Editing a long movie: what makes it hard?

The request is short. The movie is long.





A vague creative goal

Too much video to inspect repeatedly

Find the right moments without sending the whole movie each time.

VideoAgent, Sections 1–2. Cost depends on sampling, model and number of calls.

VideoAgent: a tribute to Spider-Gwen

USER REQUEST

Make a powerful short tribute to Spider-Gwen.
Capture her spirit and energy, with thrilling chases and fights.



The agent chooses the scenes and the order.

Classroom adaptation of VideoAgent's Spider-Man showcase. AI-generated teaching frames.

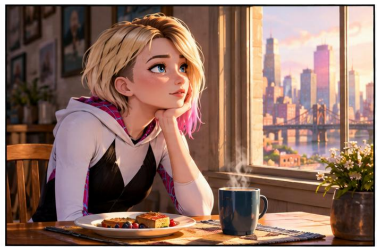
VideoRAG recap: segment the source movie

VideoAgent | **Index** | Augment | Retrieve | Trim

INPUT **A long movie (the original source)**

ORIGINAL TIMELINE **0–10s** | **10–20s** | **20–30s** …

Fixed time windows, before any editing



Source clip 3 20–30s

…



Source clip 6 50–60s

…



Source clip x 600–610s

OUTPUT **Raw source clips + their original timestamps**

Source segments remain in movie order. The agent chooses and trims final shots later. Illustrative frames and times.

VideoRAG recap: describe each clip

VideoAgent | **Index** | Augment | Retrieve | Trim

INPUT **One source clip with its original timestamps**



Source clip 6 50–60s

Sampled frames VLM → **CAPTION**

Gwen plays the drums. She wears a white hood and a black suit…

Speech audio ASR → **TRANSCRIPT, IF ANY**

Words spoken in this clip

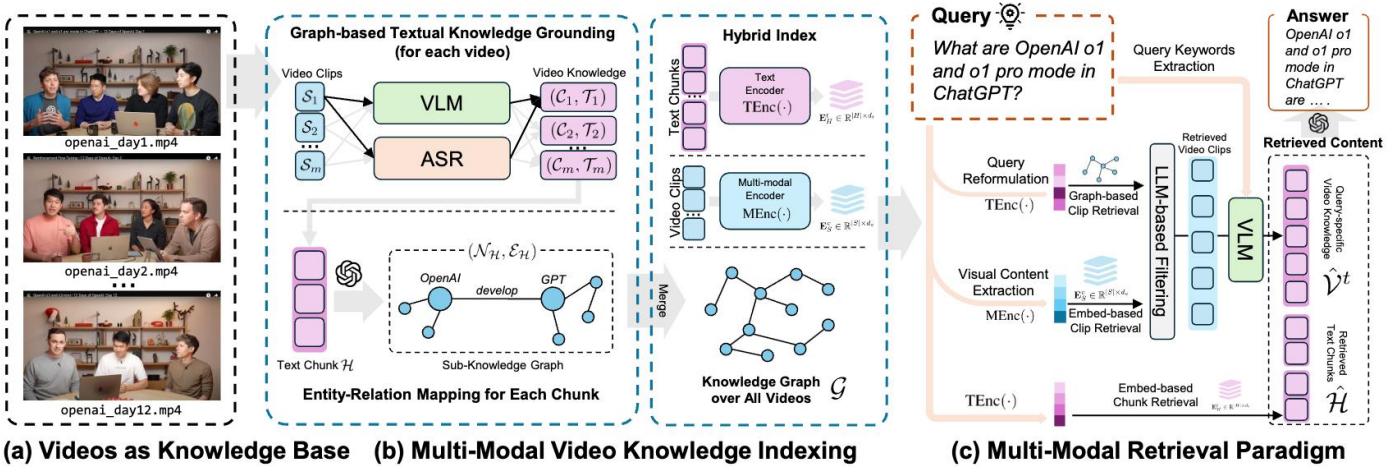
OUTPUT **Clip description + source location**

Lecture 4 VideoRAG recap. VLM describes visuals; ASR transcribes speech. Descriptions here are teaching examples.

Index the movie

VideoAgent | **Index** | Augment | Retrieve | Trim

INPUT **A long movie**



OUTPUT **Captions (MiniCPM-V 2.6*) + visual vectors (ImageBind)**

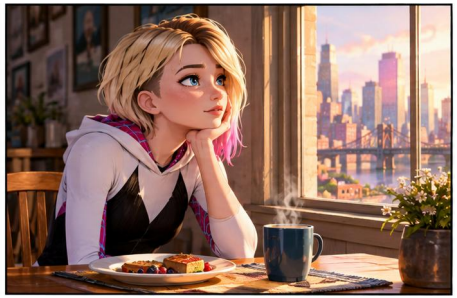
VideoRAG original framework. *VideoAgent paper setup; current code may use another VLM.

Augment the request with the movie's captions

VideoAgent | Index | **Augment** | Retrieve | Trim

INPUT **"Capture Gwen's spirit and energy"**

+ Movie captions (already created during Index)



A quiet, personal moment



Gwen drums with energy



A thrilling train chase

OUTPUT **An ordered storyboard, expressed as augmented queries**

VideoAgent shot planning. The agent grounds its choices in indexed material. Classroom query examples.

Retrieve with augmented sub-queries

VideoAgent | Index | Augment | **Retrieve** | Trim

ORIGINAL REQUEST

Capture Gwen's spirit and energy, with thrilling chases and fights.

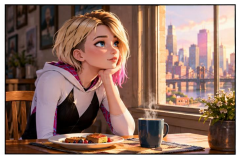
AUGMENTED VIDEO-CLIP QUERIES

Gwen sits by a sunlit window, wearing a white hood over her black suit. Warm city light frames a quiet, thoughtful moment…

Gwen plays the drums with intense energy. Purple light fills the room. Her short blonde hair moves above her white hood and black suit…

Spider-Man races along a moving train above the city. A red-and-black suit, steel tracks and nearby pursuers create a fast, tense chase…

MATCHING CLIPS







Retrieve one matching clip for each augmented sub-query.

Queries encode the planned visuals. Search compares text and video embeddings. Descriptions and frames are illustrative.

Trim the clip to fit the planned shot

VideoAgent Index Augment Retrieve **Trim**

INPUT **10-second clip + augmented video-clip query + a 3-second output slot**



VLM: choose a matching 3-second interval



OUTPUT **Keep source 34–37s. Place it at 8–11s in the final video.**

Teaching timings. 10s is the repository default; the retrieval benchmark uses 3s clips.

Recall

Did each query retrieve the correct clip?

User request (Augmented)		RETRIEVED CLIP	
A chase on a train	→	Train chase	6–9s
She sits by the window	→	By the window	0–3s
She plays the drums	→	Drumming	3–6s

Count the queries that match their labelled video clips.

Shot2Story retrieval evaluation. Colored query–clip pairs are a simplified teaching example.

EM: does the video match the query?

Embedding Matching Score

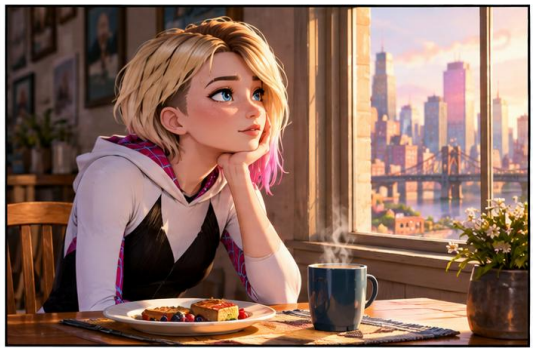
Query description: “Gwen plays the drums.”



Higher match

→ **Text–video embedding similarity** ←

Compare meaning across words and frames

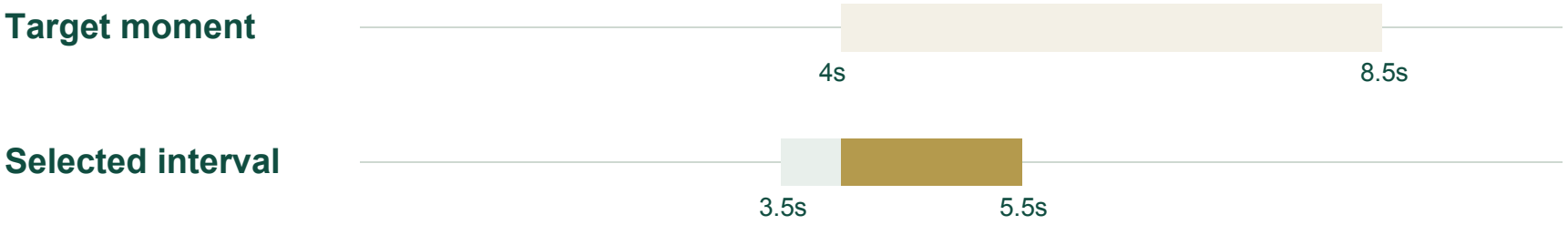


Lower match

VideoAgent Appendix A.2. Retrieved visual content and a text description. Conceptual comparison, without invented scores.

IoU: finer timing within a retrieved clip

Within the clip, how much of the target moment do we cover?



OVERLAP **1.5 seconds**

UNION **5 seconds**

IoU = overlap ÷ union = 1.5 ÷ 5 = 30%

Teaching time ranges. Gold marks the overlap. Paper evaluation uses a fixed 2s extracted interval; see Notes.

VideoAgent: retrieval results

Shot2Story: 100 videos, 599 queries

Table 2: Video understanding and retrieval performance comparison in terms of Recall@1 (%), Embedding Matching score (EM) (%), Intersection over Union (IoU) (%), API calling Costs and Time Efficiency (s).

Method	Recall	EM	IoU	Cost	Time	Method	Recall	EM	IoU	Cost	Time
Claude-Sonnet-3.7	**46.03**	27.95	23.91	0.374	43	Ours-Claude-Sonnet-3.7	44.27	**28.18**	**24.81**	0.147	37
Claude-Sonnet-3.5	27.28	27.35	12.72	0.375	41	Ours-Claude-Sonnet-3.5	**38.70**	**28.45**	**21.32**	0.147	36
Gemini-2.5-pro	45.98	27.78	**25.91**	0.349	42	Ours-Gemini-2.5-pro	**47.24**	**28.21**	25.74	0.136	37
Gemini-2.5-flash	30.04	28.04	16.69	0.070	39	Ours-Gemini-2.5-flash	**44.93**	**28.25**	**25.07**	0.028	36
GPT-4o	31.22	27.64	18.53	0.253	43	Ours-GPT-4o	**48.85**	**28.26**	**26.99**	0.099	36
VideoRAG	31.03	15.84	14.35	0.100	67	Qwen-2.5-VL-72B-Instruct	18.89	27.99	10.51	-	35
VideoMind-7B	38.26	27.75	19.67	-	21	TeaserGen	41.88	27.62	20.91	0.079	25

Recall@1
Find the moment

EM
Match the content

IoU
Match the time range

Original Table 2. 3s input segments, 1 fps. Retrieval scores, not final-video aesthetic scores.

Longer videos need more than a longer clip

Maximum duration per generation

Seedance 2.5	Kling 3.0 Omni	Gemini Omni 1.1 Flash	Wan 3.0
ByteDance	Kuaishou	Google	Alibaba
30s	**15s**	**10s**	**30s**

TARGET **A short drama or movie with a connected story**

Keep the characters, setting and narrative consistent.

Official model documentation, checked 25 Oct 2026. Single generation only; extension and stitching can produce longer videos.

ViMax: idea to video

INPUT

“A couple cooks dinner together in a cozy kitchen.”





OUTPUT **One complete video with connected shots**

ViMax Figure 1, original generated examples. This lecture follows idea-to-video.

Expand the idea into a script

ViMax | Script | Characters | Shots | Cameras | Keyframes | Video

INPUT **An idea + the user's requirements**

"A couple cooks dinner together." →

SCENE 1 · KITCHEN
They prepare dinner together.

SCENE 2 · DINING ROOM
They sit down and share the meal.

OUTPUT **A script with scenes, actions, dialogue and locations**

Classroom script. ViMax official idea2video pipeline; internal planning details are grouped here.

Create a shared character library

ViMax | Script | **Characters** | Shots | Cameras | Keyframes | Video

INPUT **Character descriptions from the script**

Character A



Character B



OUTPUT **One character record with front, side and back references**

ViMax character library. Newly generated teaching illustrations, not cropped paper frames.

Turn each scene into a shot plan

ViMax | Script | Characters | **Shots** | Cameras | Keyframes | Video

INPUT One scene script + the character library



Wide: both people in the kitchen



Over the shoulder: cooking



Medium: stirring the pan

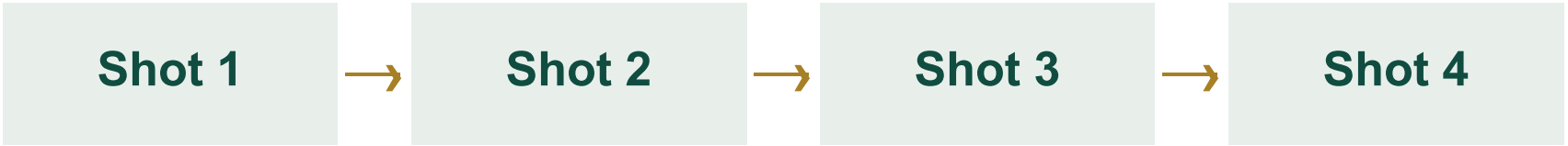
OUTPUT Ordered shots with visual and motion descriptions

Classroom shot labels. ViMax script2video pipeline and storyboard planning.

Serial generation makes every shot wait

ViMax | Script | Characters | Shots | **Cameras** | Keyframes | Video

Each new shot uses the previous shot's reference.





Four dependent shots create a long waiting chain.

Serial ablation: previous shot's first frame, not next-frame prediction. Scheduling illustration.

A camera tree shares visual references

ViMax | Script | Characters | Shots | **Cameras** | Keyframes | Video

Camera A reference image



Video model: transition A → B

Extract the new camera view



Camera B reference image

Video model: transition A → C

Extract the new camera view



Camera C reference image

B and C can run in parallel once A is ready. Reference dependencies differ from the final shot order. ViMax §2.3.

Select a keyframe for each shot

ViMax | Script | Characters | Shots | Cameras | **Keyframes** | Video

INPUT **Shot description + character portraits + camera reference**



OUTPUT **A selected keyframe for the shot**

ViMax best-of-k selection. Default k = 2 in the checked code. Generated classroom illustration.

Animate the shots and assemble the video

ViMax | Script | Characters | Shots | Cameras | Keyframes | **Video**

INPUT **Selected keyframes + each shot's motion description**



Shot 1



Shot 2



Shot 3

OUTPUT **Shot videos, joined in script order**

ViMax script2video / idea2video pipelines. Published example frames.

Consistency: compare shot features with ViCLIP

Encode each shot with ViCLIP. Average the pair similarities.

Scene A: kitchen





z1 z2 z3

Scene B: dining room





z4 z5 z6

IC

Within each scene
6 shot pairs

CC

Across scenes
9 shot pairs

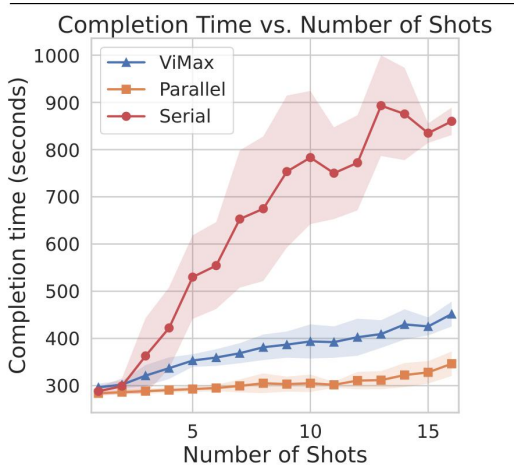
GC

All shot pairs
15 pairs in total

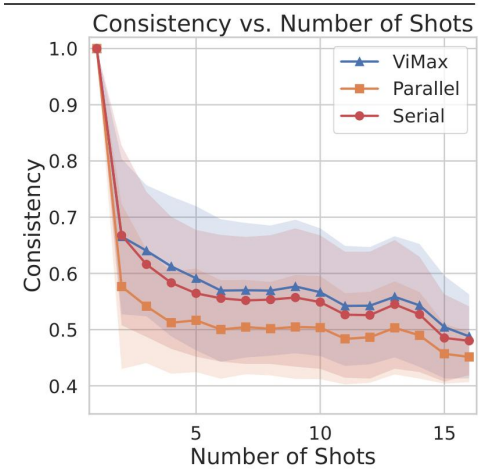
Mean pairwise cosine similarity. Two-scene grouping is illustrative. ViMax Appendix A.2.

Efficiency as the number of shots grows

Total generation time



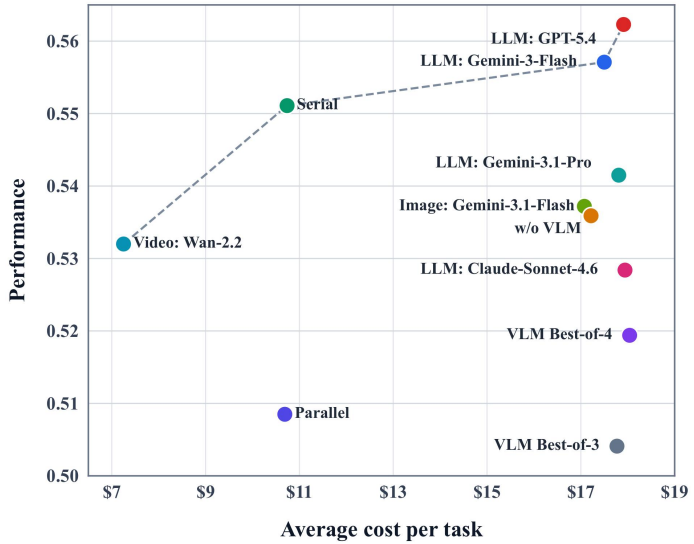
Global consistency

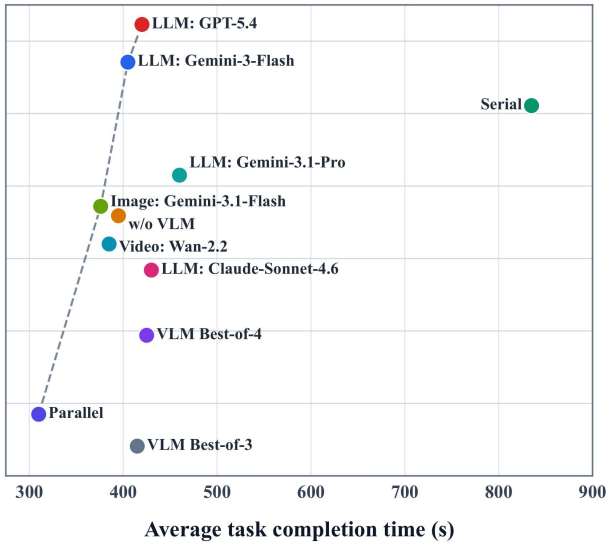


The camera tree avoids the full serial waiting chain.

Original ViMax v2 Figure 3. ViMax-Bench; curves compare scheduling strategies.

Consistency, latency and API cost





ViMax: 405s average. Serial: 835s in this benchmark.

Original ViMax v2 Figure 6. 35 stories. Costs cover API usage; latency is end-to-end.

ViMax: consistency results

ViMax-Bench: 35 stories, 438 shots

Table 1: Quantitative comparison across consistency metrics under different story-length groups. We report cross-scene consistency (CC), intra-scene consistency (IC), and global consistency (GC).

Method	Medium			Long			Overall		
	CC	IC	GC	CC	IC	GC	CC	IC	GC
Veo 3.1	0.5536	0.5640	0.5582	0.4687	0.4778	0.4704	0.4978	0.5074	0.5005
Wan2.2	0.4391	0.4462	0.4423	0.4187	0.4206	0.4188	0.4257	0.4294	0.4269
IC-LoRA-Wan2.2	0.5398	0.5713	0.5537	0.5015	0.5328	0.5087	0.5147	0.5460	0.5241
StoryDiffusion-Wan2.2	0.5606	0.5796	0.5691	0.5007	0.5195	0.5048	0.5212	0.5401	0.5269
Holocine	0.5366	0.5892	0.5597	0.4940	0.5565	0.5080	0.5098	0.5686	0.5272
MovieAgent-Gemini-3-Pro-Veo 3.1	0.4761	0.5340	0.5016	0.4410	0.5086	0.4560	0.4530	0.5174	0.4716
AniMaker-Gemini-3-Pro-Veo 3.1	0.5523	0.6014	0.5737	0.4771	0.5224	0.4875	0.5030	0.5486	0.5167
ViMax-Gemini-3-Pro-Wan2.2	0.5641	0.5895	0.5754	0.4984	0.5467	0.5093	0.5209	0.5614	0.5320
ViMax-Gemini-3.1-Flash-Veo 3.1	0.5681	0.5914	0.5785	0.5088	0.5402	0.5157	0.5291	0.5577	0.5372
ViMax-Gemini-3-Pro-Veo 3.1	**0.5886**	**0.6046**	**0.5957**	**0.5298**	**0.5629**	**0.5369**	**0.5500**	**0.5772**	**0.5571**

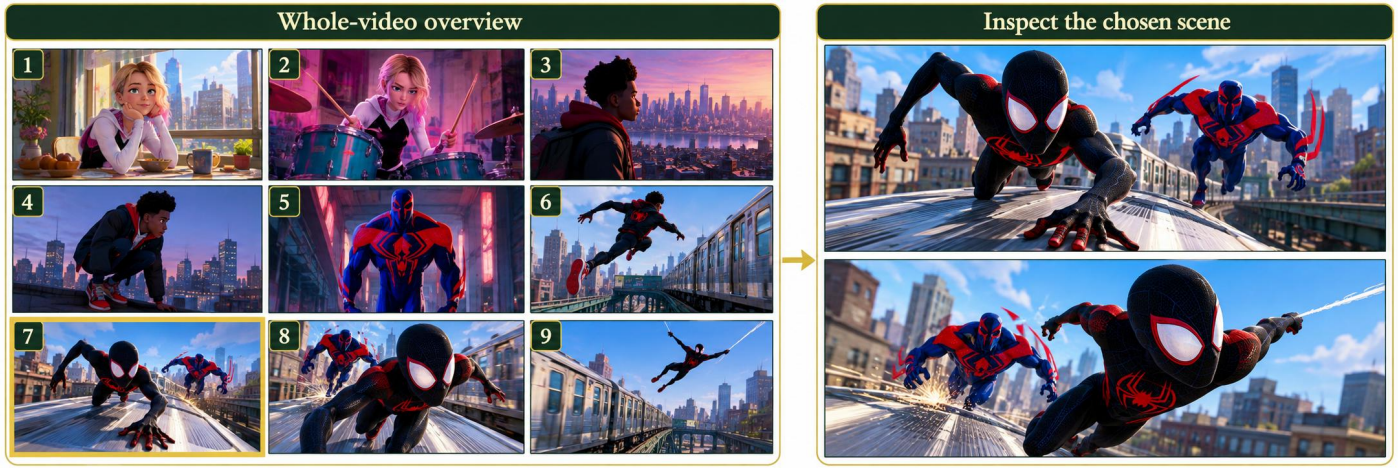
Medium: 2 scenes, 8–10 shots

Long: 3–4 scenes, 12–16 shots

Original ViMax v2 Table 1. IC / CC / GC are feature-similarity measures, not accuracy percentages.

Potential solution: inspect the video at several scales

Detect scene changes. Build a timestamped contact sheet.

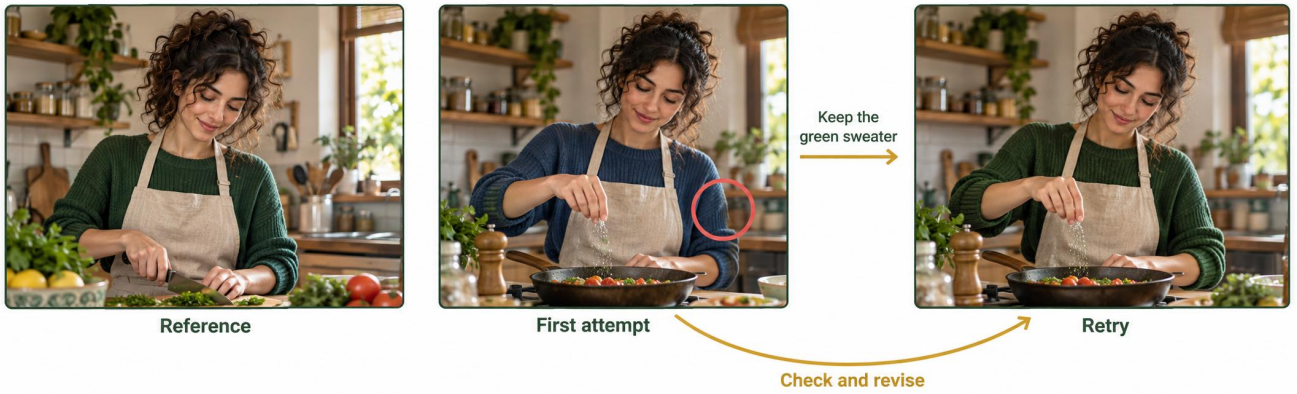


Start sparse. Inspect a promising time range more closely.

Open-ended design, not a reported benchmark result. AI-generated classroom illustration.

Potential solution: verify and refine generated shots

Inspect the image or video. Refine the prompt or visual result.



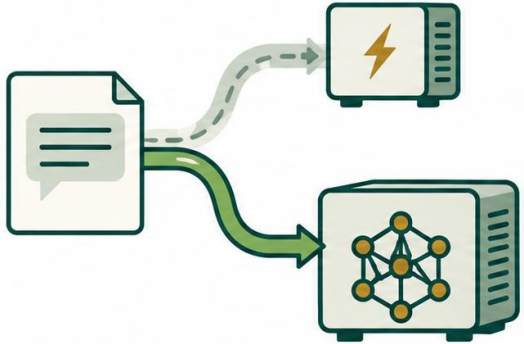
Potential benefit: better visual results

Trade-off: more time and cost

ViMax plans before rendering to reduce costly retries.

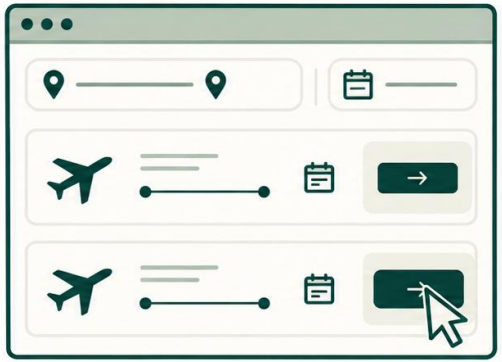
Open-ended proposal. ViMax instead emphasizes planning before expensive rendering, plus keyframe selection.

AI Agents Need Choices and Checks



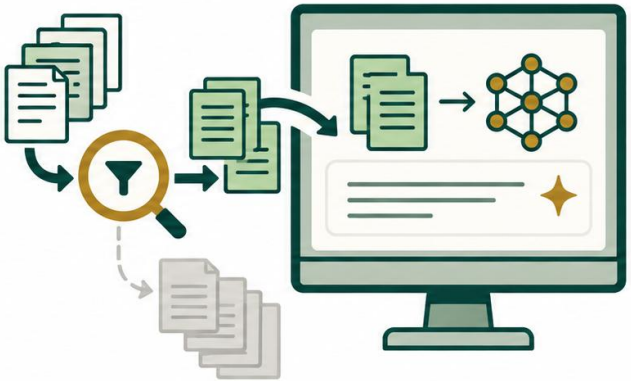
A hard request arrives.
Which model should handle it?

LLM → Call the chosen model



The flight date is wrong.
Which control should we click?

LLM → Execute the chosen action

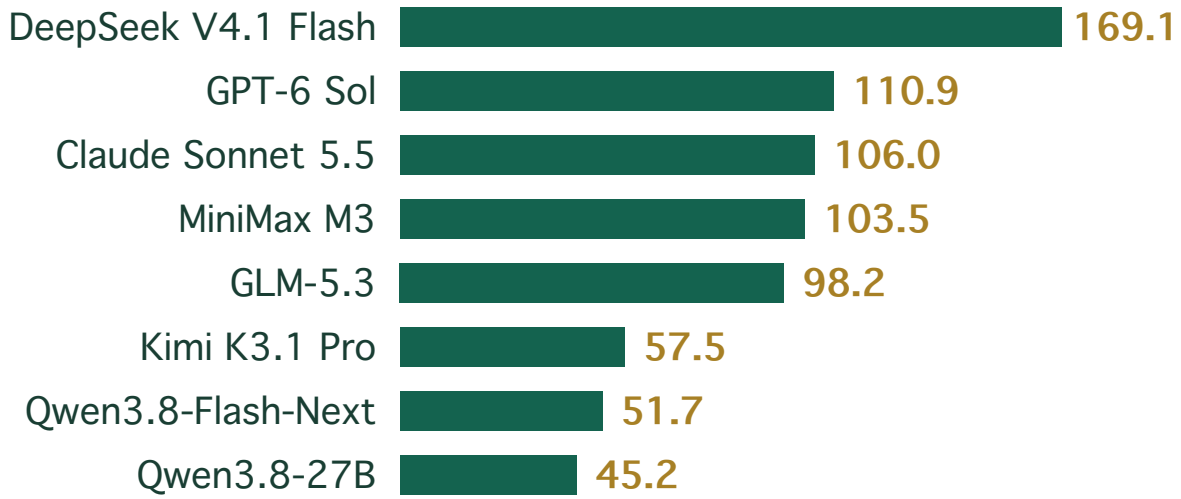


A passage was retrieved.
Does it support the answer?

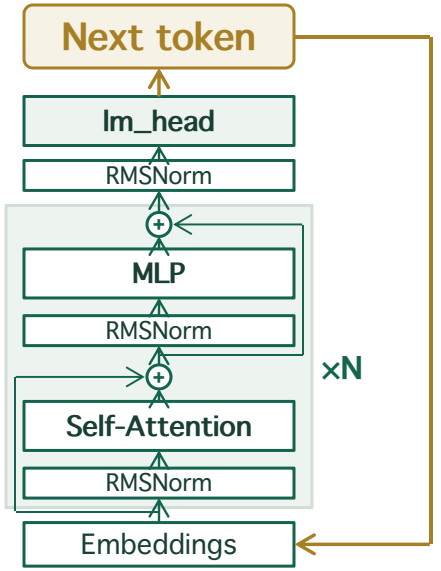
LLM → Keep or discard the evidence

Motivation 1: Low Latency

Hosted API output speed (tokens/s)



Output loop



Motivation 2: Accurate Judgments

Goal: 10 Dec, under $100

Typical RLHF

Reinforcement Learning from Human Feedback

Whole-response feedback

10 Jan selected → Fare: $29.10 → “Flight found.”

One overall preference

RLCD goal

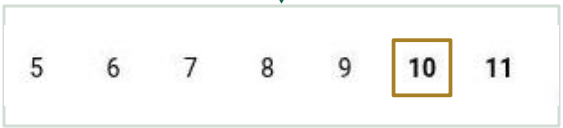
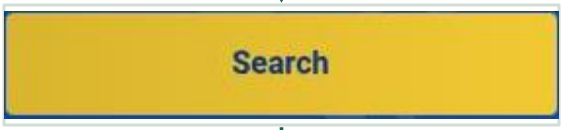
Reinforcement Learning for Calibrated Decisions

Reliable probabilities along the workflow

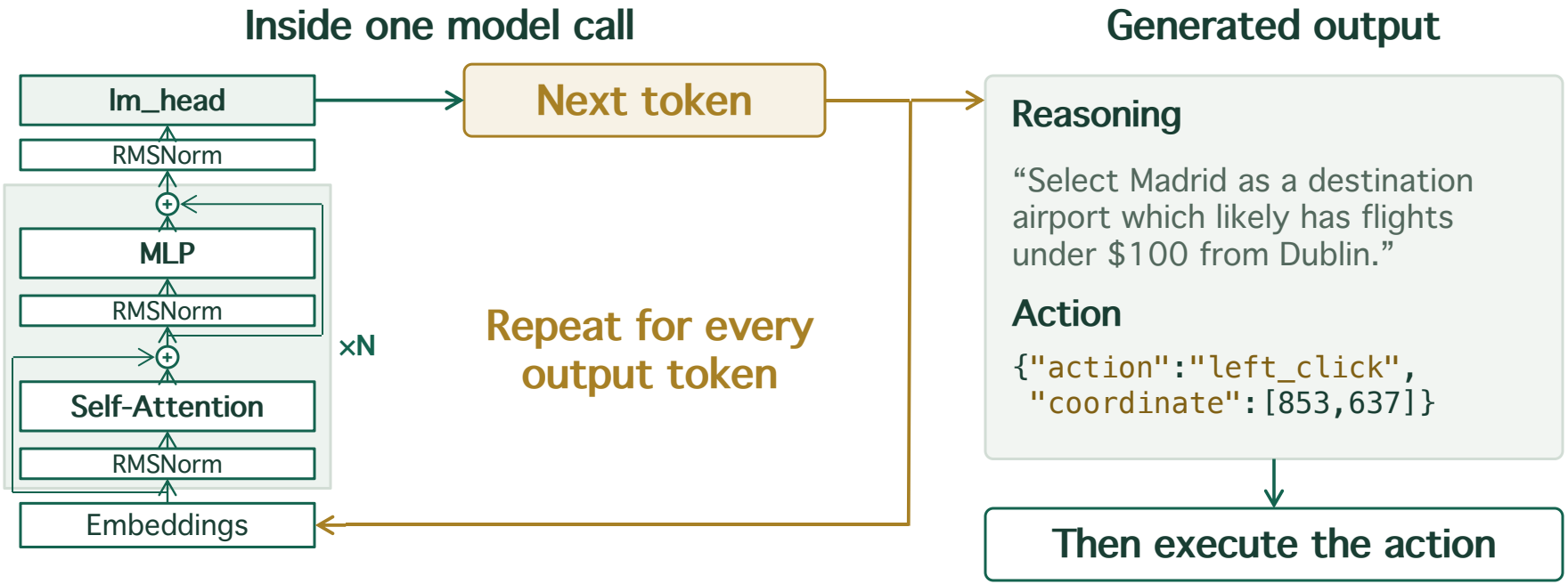
Date matches? p_1(Yes)	→	Under $100? p_2(Yes)	→	Meets request? p_3(Yes)
Checked: No		Checked: Yes		Checked: No

Example: Find a Flight

Dublin → anywhere, 10 Dec, under $100

Step 1	Choose destination		`click(853,637)`
Step 2	Choose 10 Jan		`click(885,545)`
Step 3	Search		`click(1152,270)`
Step 4	Choose Basic fare		`click(1192,685)`

Normal Generation Loop



Each output token passes through the model layers.

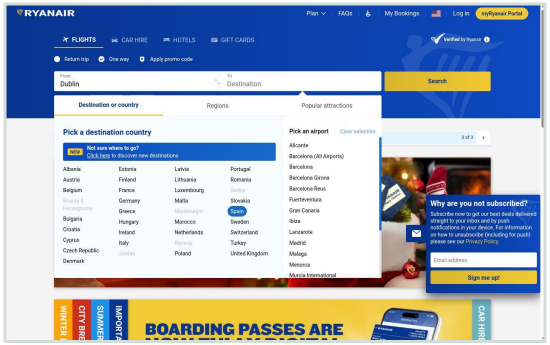
CUA: What Is the Action Space?

Operation	Compatible target / value	Example
CLICK	Point (x, y)	(853,637)
TYPE_TEXT	Editable point + text	(789,207) + text
SELECT(option)	Dropdown + option value	cabin + Economy
SCROLL	Direction + amount	DOWN, 560 px

Action: Operation and Target

4 operations with different parameters

CLICK TYPE_TEXT SELECT(option) SCROLL



1440 × 900 = 1,296,000 pixels

Large action space (raw pairs)
2 × 1,296,000 = 2,592,000

One decision model request

Split into questions over valid targets

Operation?

If CLICK: target?

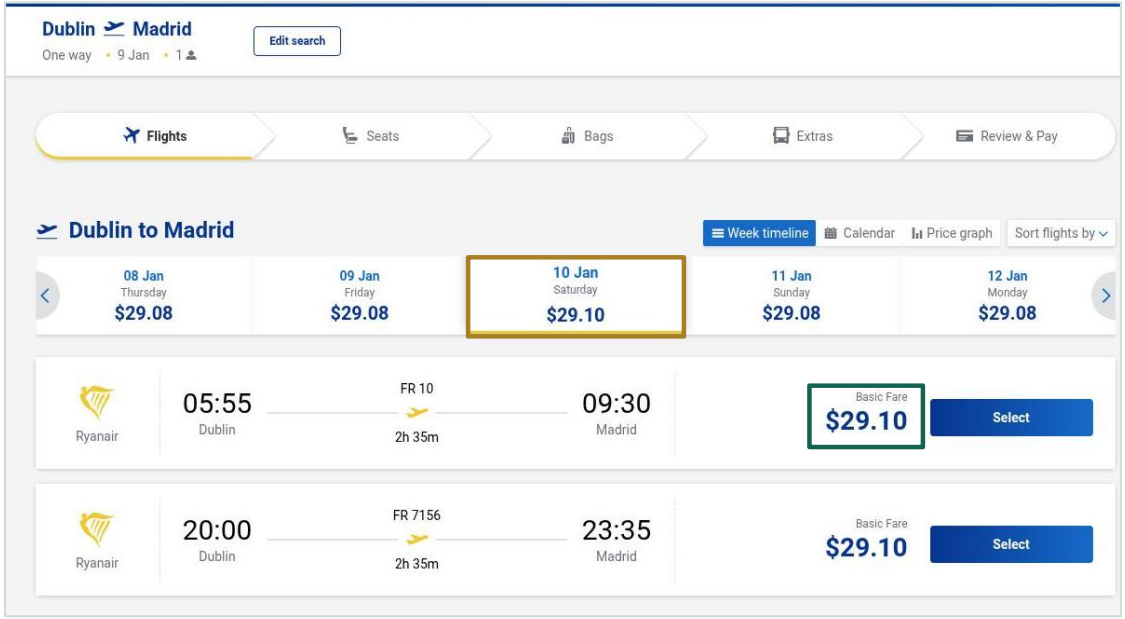
If TYPE_TEXT: editable target?

Program uses the matching target

CLICK + Madrid (853,637)

Eval: Check the Result

A flight can appear without satisfying the whole request



Flight option shown?

Time, fare and Select

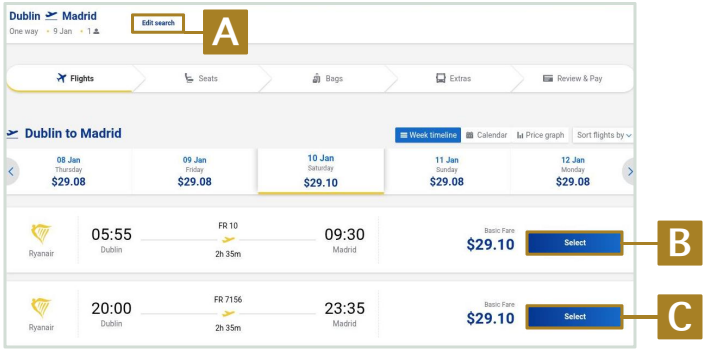
Under $100?

$29.10 < $100

On 10 Dec?

10 Jan: wrong month

Choice: Select an Option




```
questions = {"question": Choice(
  instructions="Next click?",
  criteria={
    "A": "Edit search",
    "B": "Select (05:55)",
    "C": "Select (20:00)",
  },
)}
```

A → click(414, 92)

Goal: 10 Dec, under $100

Selected date: 10 Jan. Fare: $29.10.

Question:
Which control should we click next?

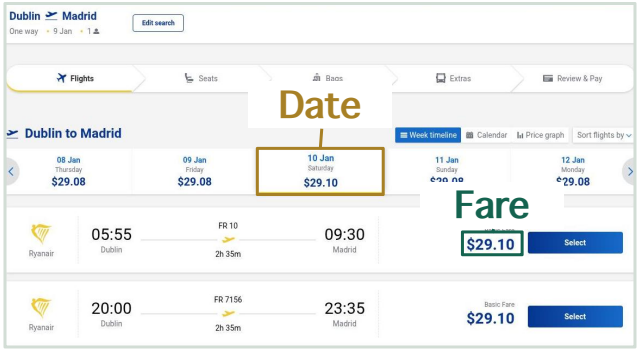
Option probabilities

A	Edit search	0.80
B	Select (05:55)	0.15
C	Select (20:00)	0.05

ID = argmax p

Chosen ID: A

Score: An Ordered Scale




```
questions = {"question": Score(
  instructions="Date and budget fit?",
  criteria=[
    "0: Wrong date",
    "1: Right date, fare >= $100",
    "2: Right date, fare < $100",
  ],
)}
```

Wrong date: this page matches level 0

Goal: 10 Dec, under $100

Date × 10 Jan Fare √ $29.10

Question:
How well does this flight fit?

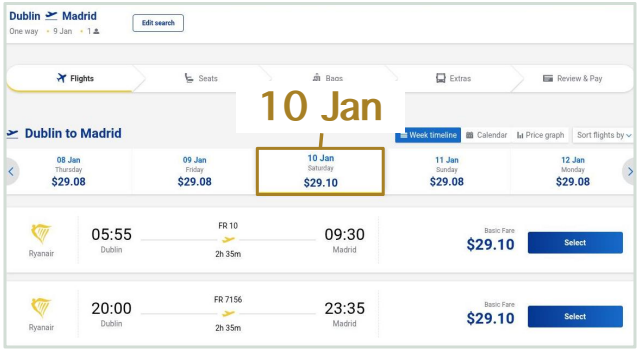
Level probabilities

0: Wrong date	0.80
1: Right date, ≥ $100	0.10
2: Right date, < $100	0.10

$$s = \Sigma_i \, i \cdot p_i$$

$$0 \times .8 + 1 \times .1 + 2 \times .1 = .3$$

Noul: Probability of Yes




```
questions = {"question": Noul(
  instructions=
    "Is departure date wrong?"
)}
```

Goal: 10 Dec, under $100

Selected: 10 Jan ≠ requested 10 Dec

Question:
Is the departure date wrong?

Probability of date error

$$p = P(\text{Yes})$$

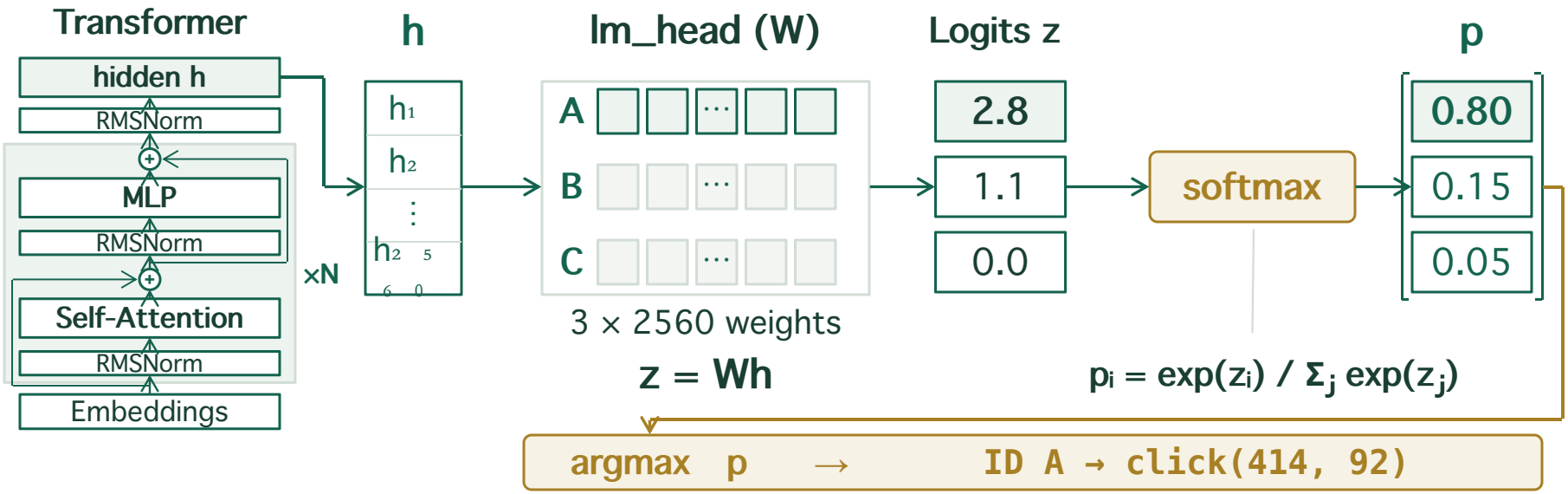
p close to 1 → date error

p close to 0 → no date error

How Does the Model Return an ID?

Score the options, then return the selected ID.

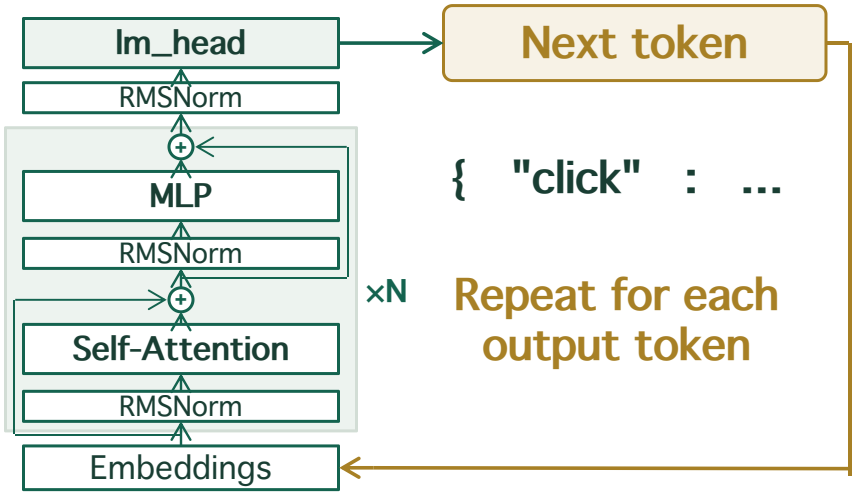
A Edit search B Select 05:55 C Select 20:00



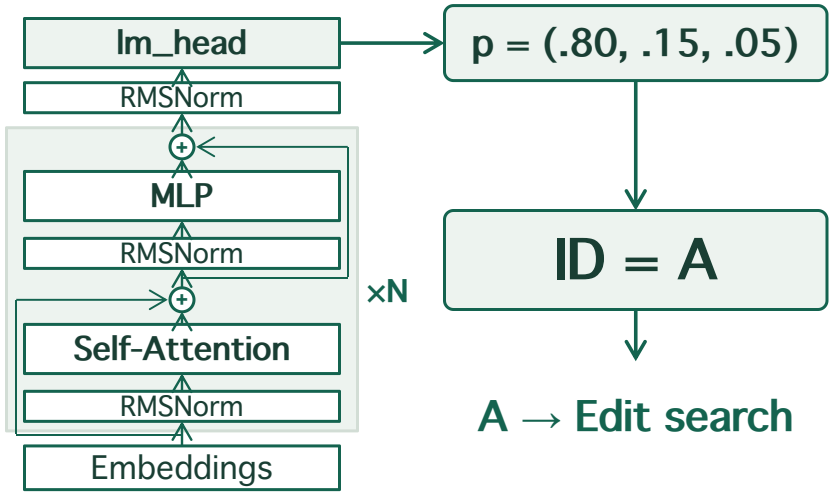
Why Fast: Return an ID

The program needs a choice ID, not a sentence.

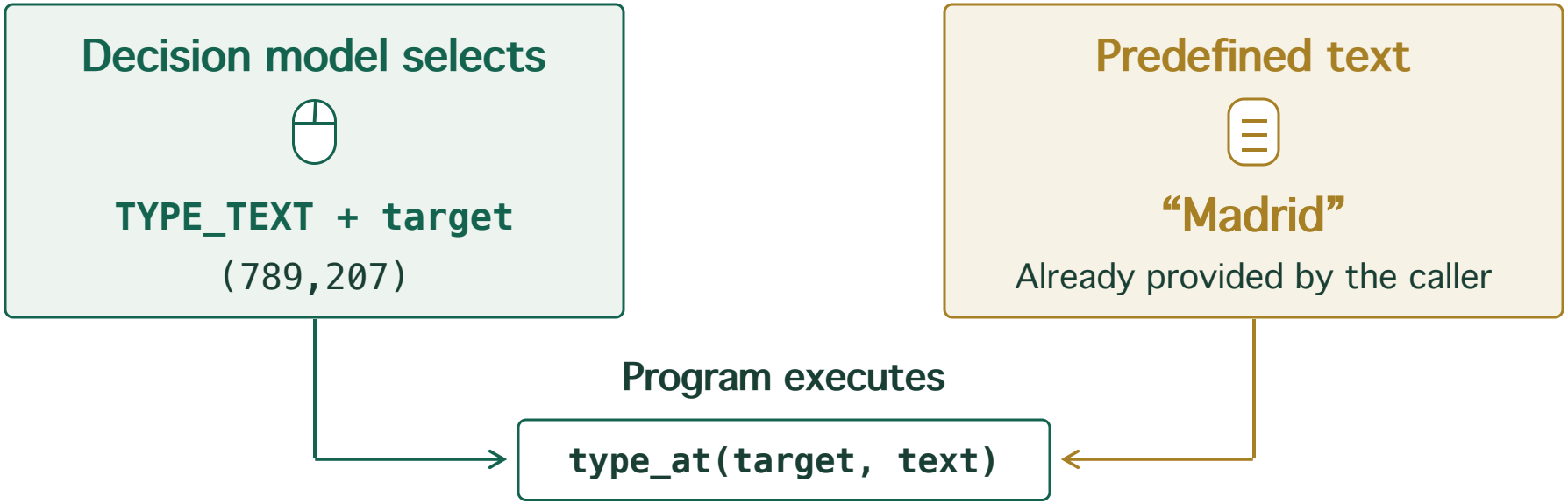
Generate the answer as text



Read the selected ID



Decision Model Cannot Generate New Text



New text needs a separate text generator.